Cron og kø: fra 326 ventende leveringer til 0

Målt og udført 8/11 2026 · site-optimering **91,3 → 92,9 %** (20,85 af 22,45 d) · S3.6 lukket, S1.2 og S4.6 delvist lukket.

Køalarmerne fra WooCommerce-integrationen var ikke bare støj. De var symptomet på, at **der ikke var noget, der kørte cron** på siden. Denne post beskriver hvordan det blev målt, hvad der blev bygget, og hvad der stadig kræver ejeren.

---

1. Det der blev målt

Måling 8/11	Før	Efter
Ventende woocommerce_deliver_webhook_async	326 (alle forfaldne)	0
Fejlede facebook_for_woocommerce_process_logs_batch	1 084	0
Aktive WooCommerce-webhooks	40 (op til 5 pr. emne mod samme URL)	29, 11 dubletter slået fra
Forældreløse cron-events	3 (bl.a. Facebook-pluginnets heartbeat)	0
sgap_daily_health sidst skrevet	7/11 20:34	8/11 14:08 (kørt af kicker'en)
Action Scheduler-handlinger i alt	7 113	6 651

Rodårsagen

1. **DISABLE_WP_CRON er sand** i `wp-config.php`.
2. **wp-cron.php kører ikke de forfaldne opgaver.** Målt med en probe: et cron-event planlagt fem sekunder ude blev «ikke» afviklet af et direkte kald (0 bytes, 200, 0,5 s), og `doing_cron`-låsen var fri. Kernen afviser selv i `_wp_cron()` når konstanten er sand.
3. **W3TC cacher siderne foran PHP.** Selv et kald til `/feed/` eller en 404-serveret side gik gennem cachen, så et `shutdown`-hook i et mu-plugin blev aldrig kørt på de sider. Bevis: et debug-felt blev først skrevet, da vi kaldte `/wp-json/` — altså en request der med sikkerhed eksekverer PHP.

Konsekvensen var ikke kun 326 webhook-leveringer: den daglige helbredsmåling, evergreen-recycling, ugentlig artikel og Action Schedulers egen kø stod stille, mens alt så "planlagt" ud i admin.

---

2. Det der blev bygget

2.1 sgap-queue-hygiene.php (mu-plugin, 9,5 KB)

Kører dagligt kl. 04:10 og gør fire ting, alle idempotente:

1. **Sletter handlinger for plugins der ikke findes** på disken (`facebook-for-woocommerce`, `all-in-one-seo-pack`, `wpforms-lite`, `mailpoet`, `woocommerce-payments`). De kan aldrig køre igen.
2. **Sletter fejlede handlinger ældre end 14 dage** — undtagen plugin'ets egne `sgap_*`.
3. **Slår dublerede webhooks fra**: af flere aktive med samme emne «og» samme URL beholdes den ældste; resten får status `disabled` (ikke slettet), så det kan rulles tilbage med ét klik.
4. **Fjerner forældreløse cron-events** og tømmer køen i op til seks batches, hvis der er over 50 ventende.

Alt skrives til optionen `sgap_queue_hygiene` med tal for hver kørsel, så effekten kan læses uden at læse koden.

2.2 sgap-cron-kicker.php (mu-plugin, 8,3 KB)

To veje til at få cron til at køre igen uden at røre `wp-config.php`:

- **På almindelige sidevisninger:** et `shutdown`-hook kører forfaldne opgaver (højst otte, højst 30 s, højst hvert andet minut). Virker kun når PHP faktisk eksekverer — det gør det på ukachede sider.
- **GET /wp-json/sgap/v1/tick:** et offentligt endpoint med samme kontrakt som `wp-cron.php` — det kører kun det, der allerede er forfaldent, og højst én gang i minuttet (transient-lås). Det er den vej der ikke afhænger af trafik eller cache.

Endpointet tåler at blive kaldt af en ur: `.github/workflows/cron-tick.yml` prikker det **hvert 15. minut**, accepterer kun `200` **med "ok":true i kroppen** og fejler jobbet med en annotation, hvis det ikke lykkes efter tre forsøg. Krops-kravet blev tilføjet efter den første kørsel, hvor et tomt 200 gik igennem som grønt.

Fælde, der kostede en runde: den første version af `tick` returnerede 200 med «tom» krop, fordi den kørte opgaverne før svaret blev sendt, og W3TC's oprydning afsluttede requesten undervejs. Det gjorde job-loggen tom — man kunne se at uret slog, men ikke hvad det ramte. Version 1.1.0 sender derfor JSON'en **før** opgaverne kører:

```
{"ok":true,"at":"2026-11-08T13:28:20+00:00","state":"running","due_before":28,"source":"tick"}
```

og skriver derefter resultatet (`ran`, `seconds`, `due_before`) i optionen `sgap_cron_kicker`. Samtidig holder `sgap_cron_kicker_history` de seneste 12 kørsler, så det kan bevises at uret tikker over tid — optionen med det seneste resultat alene kunne ikke vise det.

2.3.0 Site-planlagt tick (uafhængig af GitHub)

GitHub's `schedule` er ikke den eneste ur. Kicker 1.3.0 lægger også et **site-planlagt fem-minutters-event** (`sgap_cron_tick`, interval `sgap_five_minutes` = 300 s) i WordPress. Er eventet forfaldent når et PHP-request alligevel eksekveres, flytter kicker'en det fem minutter frem og kører de forfaldne opgaver med `source: wp-schedule` i historikken. Målt efter deploy:

```
wp_next_scheduled('sgap_cron_tick') = 2026-11-08T13:55:41+00:00
wp_get_schedule('sgap_cron_tick')   = sgap_five_minutes (300 s)
```

Det gør køen uafhængig af én ekstern scheduler: GitHub-jobbet og site-planen dækker hver sin vej, og begge skriver til `sgap_cron_kicker_history`.

Målt i drift 14:15:33 UTC — uden at nogen agent eller noget GitHub-job kaldte noget:

```
{"at":"2026-11-08T14:15:33+00:00","source":"wp-schedule","due_before":14,"ran":8,"seconds":1.03}
wp_next_scheduled('sgap_cron_tick') = 2026-11-08T14:20:32+00:00
```

Altså: den site-planlagte ur slog af sig selv, kørte otte forfaldne opgaver og satte næste kørsel. Historikken havde på det tidspunkt 12 poster fordelt på `shutdown` (10), `tick` (1) og `wp-schedule` (1).

2.3.1 Den faktiske GitHub-kørsel

Workflowen blev startet med `workflow_dispatch` (den havde ikke haft en planlagt slot endnu). Tre kørsler, alle **success**:

Run	Tid (UTC)	Udløser	Resultat	Job-log
42484917755	13:25:46 → 13:25:53	workflow_dispatch	success	Forsoeg 1: HTTP 200 (tom krop — gammel version af endpointet)

Run	Tid (UTC)	Udløser	Resultat	Job-log
42485140090	13:28:42 → 13:28:50	workflow_dispatch	success	Forsoeg 1: HTTP 200 + {"ok":true,"skipped":"locked",...}
42486836985	13:50:52 → 13:51:00	workflow_dispatch (efter krops-kravet)	success	Forsoeg 1: HTTP 200 + {"ok":true,...} — jobbet kræver nu "ok":true, så et tomt 200 ville være fejlet

Den anden kørsel ramte 60-sekunders-låsen, fordi et manuelt `curl` havde kørt 28 sekunder før — det er kontrakten (højst ét tick pr. minut), og svaret er stadig 200 med en forklaring, så jobbet ikke bliver rødt af at være for tidligt på den. Ingen secrets, tokens eller særlige rettigheder er nødvendige: jobbet kræver kun `contents: read` og et HTTPS-kald til den offentlige URL.

2.3 Målt virkning

```
tick 13:15:40 -> ran: action_scheduler_run_queue, cmplz_every_five_minutes_hook,
                 w3_pgcache_cleanup, w3_objectcache_cleanup,
                 facebook_for_woocommerce_hourly_heartbeat_cron, dbclnr_cron_tasks,
                 wc_admin_process_orders_milestone, wc_admin_unsnooze_admin_notes
                 due_before: 18   seconds: 0.11
```

Efter kørslen: **pending = 0**, Facebook-handlinger **0**, webhook-dubletter **11 slået fra**, cron-events for det fjernede Facebook-plugin **0**. Hygiejnen kørt to gange i træk giver `orphans_deleted: 0, duplicates_disabled: 0` — den er idempotent.

3. De to roadmap-punkter der flyttede sig

S1.2 — 0,2 af 0,5 d

Præmissen var forældet: **940 og 941 er allerede publiceret** og målt live (`/product/clockwork-seer-pendant/` og `/product/the-aeronaut-pocket-watch/`, begge 200 med `Product` -schema og priserne 129/149). De øvrige fire — **942, 765, 766 og 767** — lå i papirkurven efter S1.4-oprydningen og er nu **gendannet som kladder** med pris, SKU, beskrivelse (975–1 376 tegn), kategori og SEO-titel/-beskrivelse intakt. Det der mangler, er ikke arbejde: det er ejerens **ja/nej pr. produkt** til at sælge dem.

S4.6 — 0,1 af 0,4 d

Cache-bekymringen er **afkræftet med måling**: `/cart/`, `/checkout/` og `/my-account/` svarer alle med `cache-control: no-cache, must-revalidate, max-age=0, private`, så W3TC ikke kan levere en cachet kurv — uden at W3TC's config blev rørt.

Det der «ikke» er i orden, er forsendelsen: **kun én zone (Danmark)** med `free_shipping` og **intet minimum**, og **ingen zone for EU/UK/RoW**. En kunde uden for Danmark får ingen forsendelsesmetode i checkout. Det kræver ejerens priser og zoner — ikke kode.

4. Hvad der stadig kræver ejeren

Punkt	Præcis mangel
AIMLAPI-kredit	Sidste generering 7/11 fejlede med *»You've run out of funds«* — der laves ingen nye kladder før kontoen er fyldt op
sgap_alert_email	Feltet er tomt; fejl registreres i sgap_health, men ingen mail går ud
Full auto vs. godkendelse	sgap_full_auto_enabled = no, og 40 elementer venter på godkendelse

Punkt	Præcis mangel
Distributionsvej	publish_mode = rss, 0 Publer-konti, Pinterest 401, fem kanaler uden token
S1.2	Ja/nej pr. produkt til 942, 765, 766, 767
S4.6	Zoner (EU/UK/RoW), fragtklasser og fri fragt-grænse

De fire øverste står nu i `blocked_by` på plugin-projektet i `projektstatus.json`, så de også står i `docs/projektstatus.md` §2.

2.3.2 Tilstanden efter ticks (målt 13:32 UTC)

Måling	Værdi
Ventende woocommerce_deliver_webhook_async	0
Handlinger med facebook_for_woocommerce%	0
WooCommerce-webhooks	29 aktive / 11 disabled (wp_wc_webhooks, WC 11's egen tabel)
Køhygiejne kørt to gange i træk	orphans_deleted: 0, stale_deleted: 0, duplicates_disabled: 0, cron_events_removed: 0, drained: 0 — idempotent
Køhygiejne næste kørsel	2026-11-09T04:10:00Z, recurrence daily
Tick-historik	13:28:20 tick (28 forfaldne, 12 kørt), 13:28:29 shutdown (8), 13:29:53 shutdown (8), 13:32:24 tick (31, 12), 13:32:35 shutdown (8)

At der både står `tick` og `shutdown` i historikken er pointen: uret slår nu ad to veje — GitHub-jobbet og de PHP-requests der alligevel eksekveres — så køen ikke afhænger af én af dem.

5. Hvad der ikke er bevist

- **Den planlagte (schedule) kørsel er ikke observeret endnu** — kun `workflow_dispatch`-kørslerne ovenfor. Workflowen står `active` med `cron: '*/15 * * * *'`, men ved måling 13:49 UTC (20 minutter og to 15-minutters-slots efter oprettelsen) havde GitHub ikke startet nogen `schedule`-kørsel. Nye `schedule`-definitioner lægges i kø med forsinkelse, og GitHub garanterer ikke minuttet — derfor er det her kun bevist at mekanismen virker (jobbet grønt, endpointet 200 med JSON), ikke at uret allerede har slået af sig selv gennem GitHub.
- **Køen hænger dog ikke på den ene vej.** `sgap_cron_kicker_history` viser `shutdown`-kørsler ind imellem `tick`-kørslerne (fx 13:29:53 og 13:32:35), altså at forfaldne opgaver også blev kørt af almindelige PHP-requests uden at nogen agent kaldte endpointet. Og selve køhygiejnen er planlagt **inde i WordPress** (dagligt 04:10), ikke i GitHub.
- **At wp-cron.php er død er målt, men ikke forklaret** på host-niveau. Det kan være Simplys egen blokering eller en WAF-regel. Kicker'en gør forklaringen unødvendig, men hvis webhotellet senere retter det, kører begge veje parallelt uden skade (begge er låst til ét kald pr. minut).
- **Webhook-dubletterne kan komme igen**, hvis den eksterne tjeneste genopretter dem. Hygiejnen kører dagligt og slår dem fra igen; den er ikke en permanent fjernelse.